# Chapter 2. Managing threads

**This chapter covers**

- Starting threads, and various ways of specifying code to run on a new thread
- Waiting for a thread to finish versus leaving it to run
- Uniquely identifying threads

OK, so you've decided to use concurrency for your application. In particular, you've decided to use multiple threads. What now? How do you launch these threads, how do you check that they've finished, and how do you keep tabs on them? The C++ Standard Library makes most thread-management tasks relatively easy, with just about everything managed through the `std::thread` object associated with a given thread, as you'll see. For those tasks that aren't so straightforward, the library provides the flexibility to build what you need from the basic building blocks.

In this chapter, I'll start by covering the basics: launching a thread, waiting for it to finish, or running it in the background. We'll then proceed to look at passing additional parameters to the thread function when it's launched and how to transfer ownership of a thread from one `std::thread` object to another. Finally, we'll look at choosing the number of threads to use and identifying particular threads.

## 2.1. Basic thread management

Every C++ program has at least one thread, which is started by the C++ runtime: the thread running `main()`. Your program can then launch additional threads that have another function as the entry point. These threads then run concurrently with each other and with the initial thread. Just as the program exits when the program returns from `main()`, when